

1) Averiguar dispersiones x, y y ver tema circularidad

Tu detector ya calcula casi todo esto. Una vez que selecciona los píxeles que considera pertenecientes al objeto, calcula su centroide:

$$\bar{x} = \frac{1}{N} \sum_i x_i, \quad \bar{y} = \frac{1}{N} \sum_i y_i$$

y después:

$$\sigma_x^2 = \frac{1}{N} \sum_i x_i^2 - \bar{x}^2$$

$$\sigma_y^2 = \frac{1}{N} \sum_i y_i^2 - \bar{y}^2.$$

Eso corresponde en el código a **varX** y **varY**.

Ahora mismo, sin embargo, imprimís solamente una **sigma** combinada:

$$\sigma = \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}}$$

Modificar la salida para mirar **por separado** σ_x y σ_y :

```
float sigmaX = sqrtf(ball.varX);
```

```
float sigmaY = sqrtf(ball.varY);
```

```
Serial.printf(
```

```
"x:%.1f\ty:%.1f\tN:%lu\tsigmaX:%.2f\tsigmaY:%.2f\tC:%.2f\tfps:%.1f\n",
```

```
ball.x,
```

```
ball.y,
```

```
ball.pixels,
```

```
sigmaX,
```

```
sigmaY,
```

```
ball.circularity,
```

```
fps
```

```
);
```

→ Determinar **qué forma tiene la nube de píxeles detectados**.

Para una pelotita aproximadamente circular esperaríamos, idealmente,

$$\sigma_x \approx \sigma_y.$$

Se define actualmente la circularidad como $C = \frac{\max(\sigma_x^2, \sigma_y^2)}{\min(\sigma_x^2, \sigma_y^2)}$

Un objeto perfectamente isotrópico tendría $C \approx 1$ y, cuanto más alargada sea la detección, mayor será C .

Pero hay algo importante: actualmente la circularidad **no está rechazando detecciones**. La condición está comentada:

```
result.valid =
```

```
    N >= nmin && N <= nmax; // && result.circularity <= circularityMax;
```

→ Primero medir C con la pelota real en muchas posiciones y también sin pelota; después elegimos umbral razonable. No poner `circularityMax = 1.5` a ciegas.

2) Consultar mediciones hechas con sensor

Comparar la medición de la cámara con las mediciones hechas con el otro sensor (el HC-SR04).

Sirve para poner el carrito quieto en posiciones conocidas y registrar simultáneamente: x_{cam} , x_{sensor} ,

$x_{referencia}$. IMPORTANTE: dos experimentos.

- Primero, **calibración**: usamos posiciones físicas conocidas para obtener la transformación píxeles → centímetros.
- Después, **validación**: usamos otras posiciones que no participaron de la calibración y comparamos cuánto mide la cámara respecto de la referencia y/o del otro sensor.

Así podemos obtener error: $e_i = x_{cámara,i} - x_{ref,i}$, métricas como sesgo, desviación estándar y RMSE para justificar cuantitativamente qué tan buena es la cámara como sensor de posición.

3) Cómo hacer para transformar la posición x de píxeles a cm considerando que no sé exactamente hacia adónde apunta la cámara

No es necesario conocer exactamente la orientación de la cámara. Con la cámara colocada en una posición definitiva, marcamos varias posiciones conocidas sobre la barra, por ejemplo: $-15, -10, -5, 0, 5, 10, 15$ cm y para cada una registramos el x en píxeles. Obtendríamos algo del estilo:

$x_{real}(cm)$	$x_{cámara}(pixel)$
-15	p_1
-10	p_2
-5	p_3
0	p_4
5	p_5

10

 p_6

15

 p_7

Ajustamos: $x_{cm} = a * x_{px} + b$, donde b absorbe que la cámara no esté centrada. Incluso si la imagen está invertida, no hay problema: simplemente obtendríamos $a < 0$. Si la cámara está bastante inclinada respecto de la barra, puede aparecer **perspectiva** y la relación dejar de ser perfectamente lineal. Por eso es importante usar varios puntos y no solamente los dos extremos. Luego, graficamos: x_{real} vs x_{px} , mirando los residuos del ajuste lineal. Si la recta funciona bien, nos quedamos con ella. Si aparece una desviación sistemática, ahí evaluamos una corrección proyectiva.

Podríamos hacer **el experimento todo junto...**

Fijando la cámara y hacinedo, por ejemplo, **7 posiciones conocidas** del carrito. En cada posición dejamos el objeto quieto unos segundos y guardamos x , y , N , σ_X , σ_Y y C . Si podemos, anotamos también la lectura del HC-SR04. Con **un solo ensayo** obtenemos datos para las tres tareas: vemos las dispersiones y circularidad, calibramos píxeles $\rightarrow$ cm y comparamos cámara contra el otro sensor. Y después hacemos 3–5 posiciones nuevas **solo para validación**, que no usamos para calcular a y b .